

**«Санкт-Петербургский государственный электротехнический университет  
«ЛЭТИ» им. В.И.Ульянова (Ленина)»  
(СПбГЭТУ «ЛЭТИ»)**

## **ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: UI-тестирование**

Студент гр. 4341  
Студент гр. 4341  
Студент гр. 4343  
Студент гр. 4345

\_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_

Д.Д. Скрипникова  
В.Г. Филатов  
Р.Б. Едыгеев  
Н.С. Лунченко

Руководитель

\_\_\_\_\_

А.М. Шевелёва

Санкт-Петербург  
2026

## **ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ**

Студент: Н.С. Лунченко

Группа: 4341

Тема практики: UI-тестирование

Задание на практику:

Разработать набор автоматизированных UI-тестов для веб-приложения на языке Java с использованием технологий Maven, JUnit 5, Selenide и паттерна Page Object Model.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

|              |       |               |
|--------------|-------|---------------|
| Студент      | _____ | Н.С. Лунченко |
| Руководитель | _____ | А.М. Шевелёва |

## **АННОТАЦИЯ**

Отчёт по летней учебной практике содержит описание разработанного набора автоматизированных UI-тестов для веб-приложения GitHub.

В рамках работы проведён анализ предметной области, обоснован выбор технологического стека: Java 17 [1], Maven [2], JUnit 5 [3], Selenide [4]. Описана архитектура тестового проекта, построенная на основе паттерна Page Object Model.

Разработано 8 автоматизированных тестов, покрывающих создание публичного и приватного репозитория, создание и удаление файлов, создание и удаление веток, создание и закрытие Pull Request'ов.

В отчёте приведены описание архитектуры проекта с UML-диаграммой, а также изложены результаты выполнения работы.

## **SUMMARY**

The Summer internship report contains a description of the developed set of automated UI tests for the GitHub web application.

As part of the work, an analysis of the subject area was carried out, and the choice of a technology stack was justified: Java 17, Maven, JUnit 5, Selenide. The architecture of the test project based on the Page Object Model pattern is described.

8 automated tests have been developed covering the creation of public and private repositories, the creation and deletion of files, the creation and deletion of branches, the creation and closure of Pull Requests.

The report describes the architecture of the project with a UML diagram, as well as the results of the work.

## СОДЕРЖАНИЕ

|                                                                                        |                                     |
|----------------------------------------------------------------------------------------|-------------------------------------|
| ВВЕДЕНИЕ .....                                                                         | 6                                   |
| 1 ПОСТАНОВКА ЗАДАЧИ .....                                                              | 7                                   |
| 1.1 Создание публичного репозитория .....                                              | 7                                   |
| 1.2 Создание приватного репозитория.....                                               | 8                                   |
| 1.3 Создание файла .....                                                               | 8                                   |
| 1.4 Удаление файла .....                                                               | 10                                  |
| 1.5 Создание ветки .....                                                               | 12                                  |
| 1.6 Удаление ветки .....                                                               | 14                                  |
| 1.7 Создание Pull Request.....                                                         | 15                                  |
| 1.8 Закрытие Pull Request.....                                                         | 18                                  |
| 1.9 Результат выполнения тестов .....                                                  | 19                                  |
| 2 РЕЗУЛЬТАТЫ РАБОТЫ .....                                                              | 21                                  |
| 2.1. Формулировка задачи 1 (Элементы) .....                                            | 21                                  |
| 2.2. Формулировка задачи 2 (Страницы) .....                                            | 23                                  |
| 2.3. Формулировка задачи 3 (Тесты).....                                                | 27                                  |
| 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ .....                                                        | 31                                  |
| 3.1. Тестирование управления ветками (BranchTests) <b>Error! Bookmark not defined.</b> |                                     |
| 3.2. Тестирование управления Pull Request'ами (PullRequestTests) .....                 | <b>Error!</b>                       |
| <b>Bookmark not defined.</b>                                                           |                                     |
| 3.3. Процесс дебага и исправления ошибок.....                                          | <b>Error! Bookmark not defined.</b> |
| 4 ОТЗЫВ РУКОВОДИТЕЛЯ.....                                                              | 36                                  |
| ЗАКЛЮЧЕНИЕ .....                                                                       | 37                                  |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                                                 | 38                                  |

## ВВЕДЕНИЕ

В рамках летней учебной практики по направлению «UI-тестирование» перед командой стояла задача разработать набор автоматизированных UI-тестов для веб-приложения на языке Java с использованием технологий Maven, JUnit 5, Selenide и паттерна Page Object Model.

Объект тестирования: веб-платформа GitHub [5].

Предмет тестирования: функциональный блок «Управление репозиториями» (создание, удаление репозитория, работа с файлами, ветками, Pull Request'ами).

Вклад каждого участника:

- Скрипникова Дарья: разработка элементов страниц, создание иерархии базовых и составных элементов организации структуры проекта, настройка логирования
- Лунченко Никита: реализация страниц авторизации, управления репозиториями и файлами, интеграция элементов в страницы.
- Филатов Вячеслав и Едыгеев Руслан: разработка тестов, написание и отладка тестов для проверки функциональности работы с репозиториями, файлами, ветками и Pull Request'ами.

# 1 ПОСТАНОВКА ЗАДАЧИ

## 1.1 Создание публичного репозитория

Цель: проверить возможность создания нового репозитория с публичной видимостью.

Ожидаемый результат: репозиторий создаётся, отображается в списке репозиториях пользователя и имеет статус «Public».

На рисунках 1-2 (рис. 1 - рис. 2) представлены шаги выполнения теста.

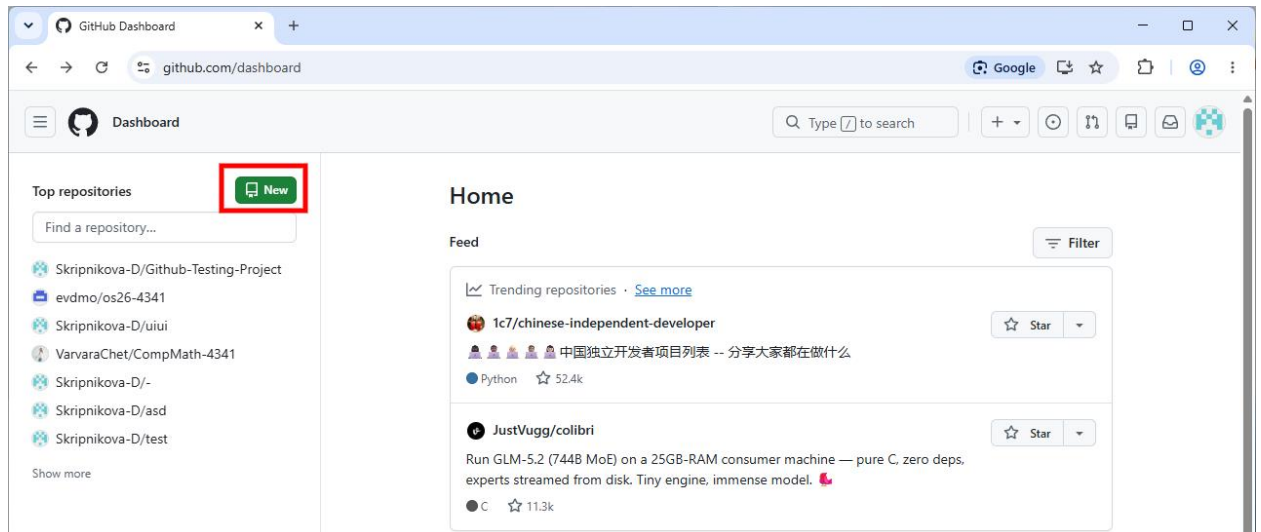


Рисунок 1 – Создание нового репозитория. Часть 1

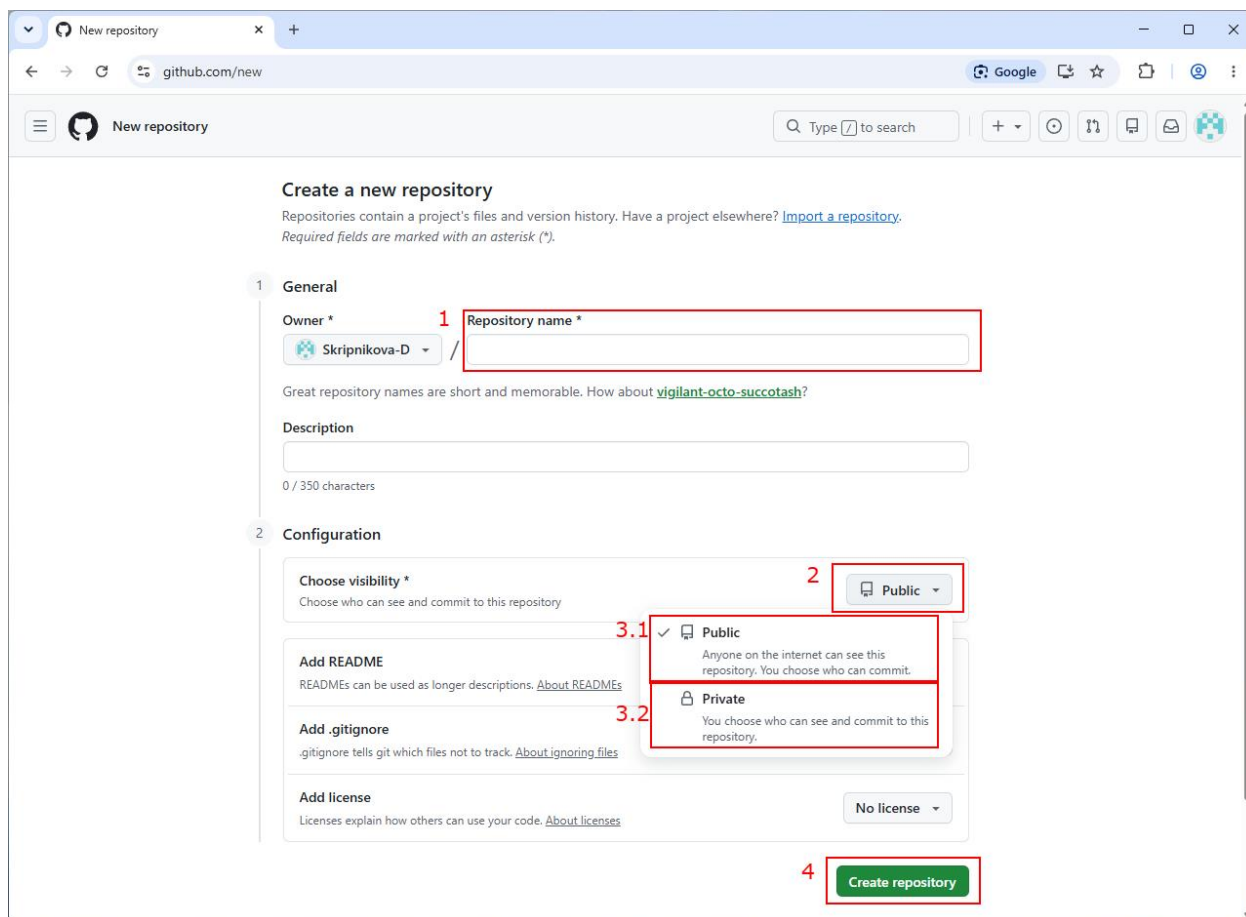


Рисунок 2 – Создание нового репозитория. Часть 2

## 1.2 Создание приватного репозитория

Цель: проверить возможность создания нового репозитория с приватной видимостью.

Ожидаемый результат: репозиторий создаётся, отображается в списке репозиториях пользователя и имеет статус «Private».

Шаги выполнения теста показаны на рисунках 1 - 2 (см. выше) с поправкой на то, что для создания приватного репозитория необходимо следовать шагу 3.2 (рис. 2), а не 3.1

## 1.3 Создание файла

Цель: проверить возможность создания нового файла через веб-интерфейс GitHub.

Ожидаемый результат: файл создаётся и отображается в списке файлов репозитория.

Шаги выполнения теста показаны на рисунках 3 - 5 (рис. 3, рис. 4, рис. 5)

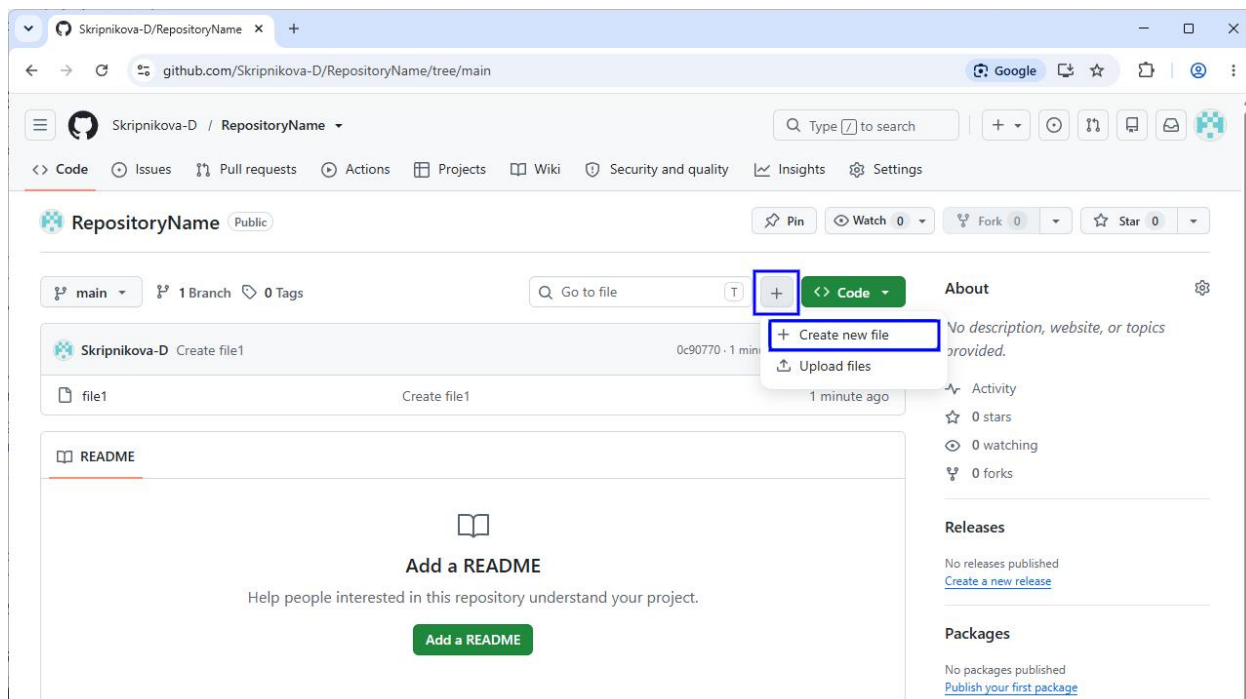


Рисунок 3 – Создание нового файла. Часть 1

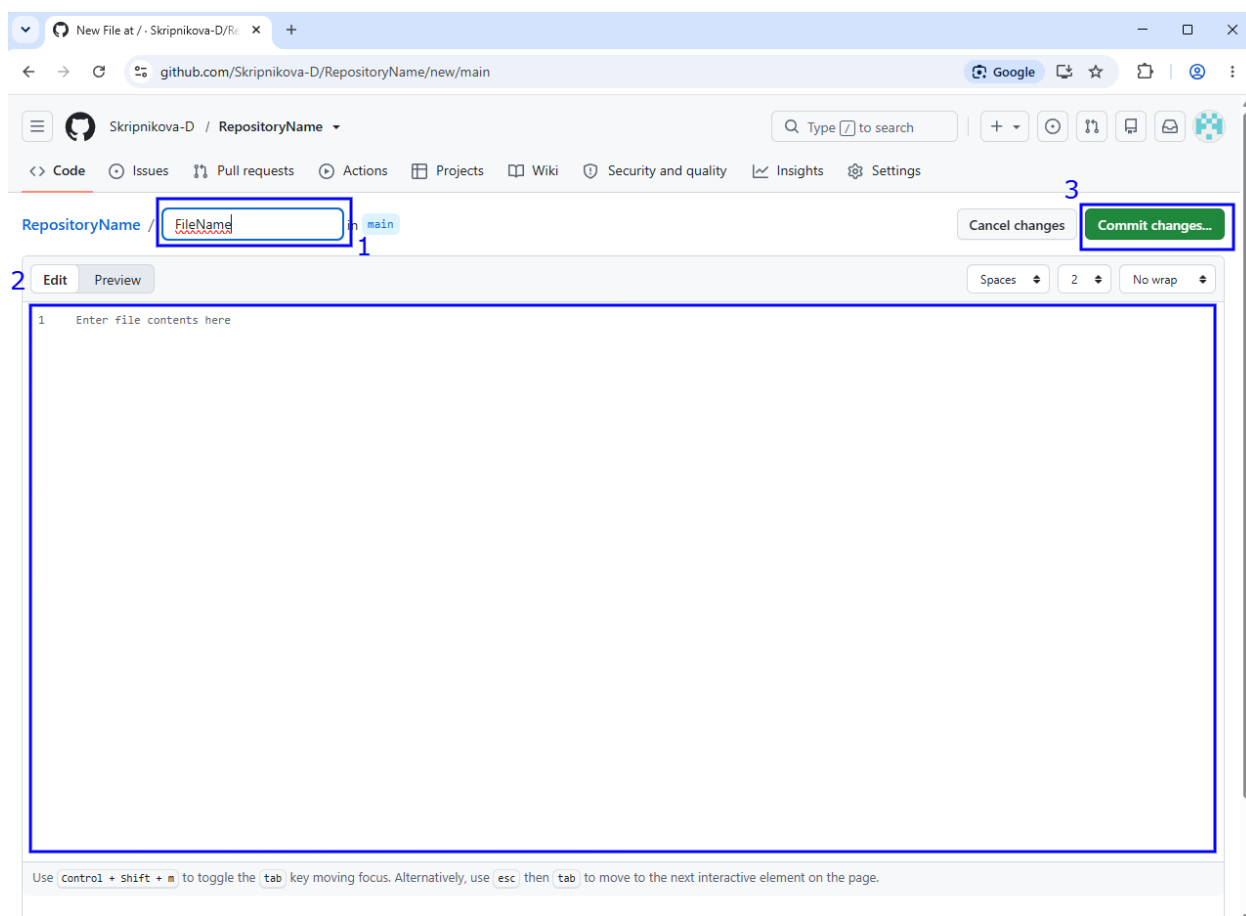


Рисунок 4 – Создание нового файла. Часть 2

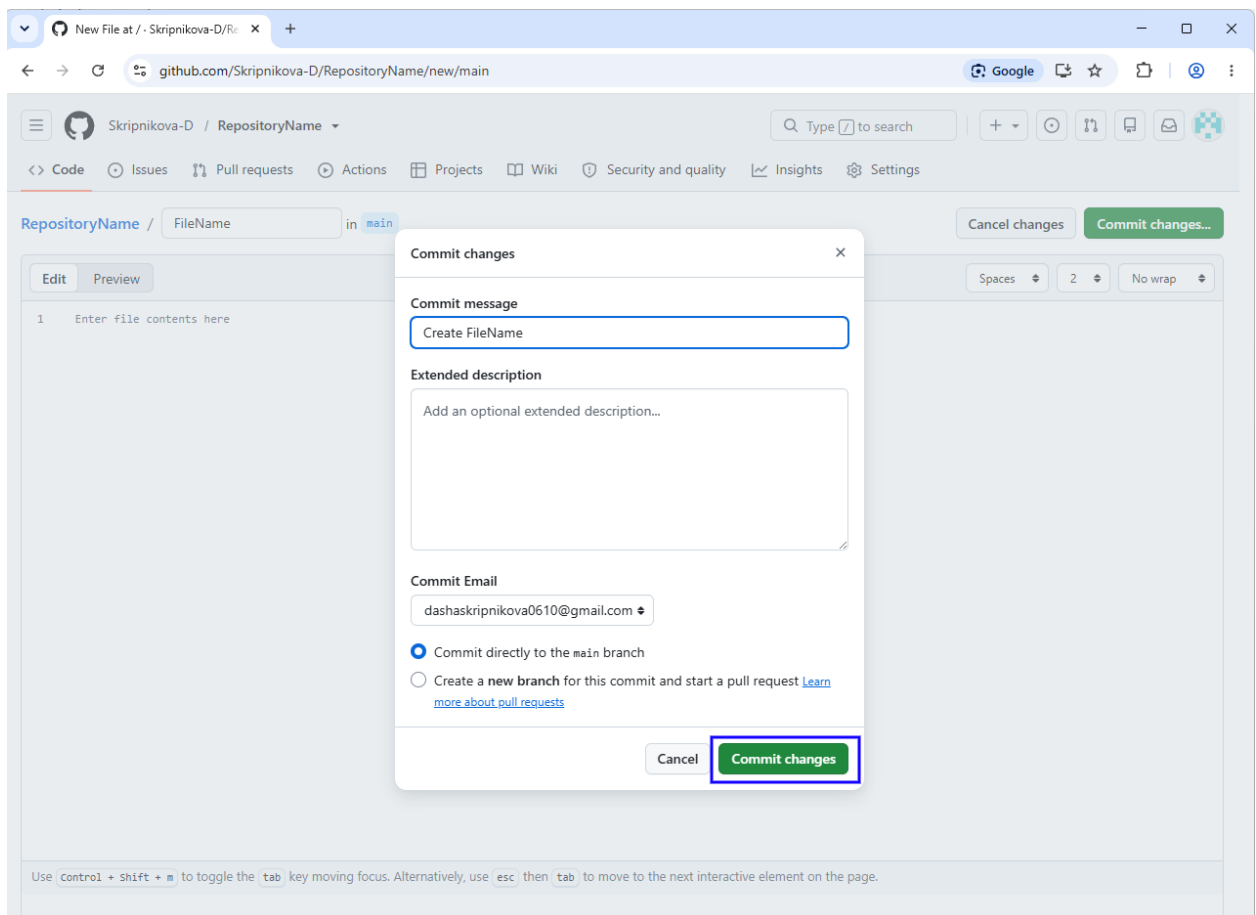


Рисунок 5 – Создание нового файла. Часть 3

## 1.4 Удаление файла

Цель: проверить возможность удаления существующего файла.

Ожидаемый результат: файл удаляется и не отображается в списке файлов.

Шаги выполнения теста показаны на рисунках 6 - 9 (рис. 6, рис. 7, рис. 8, рис. 9)

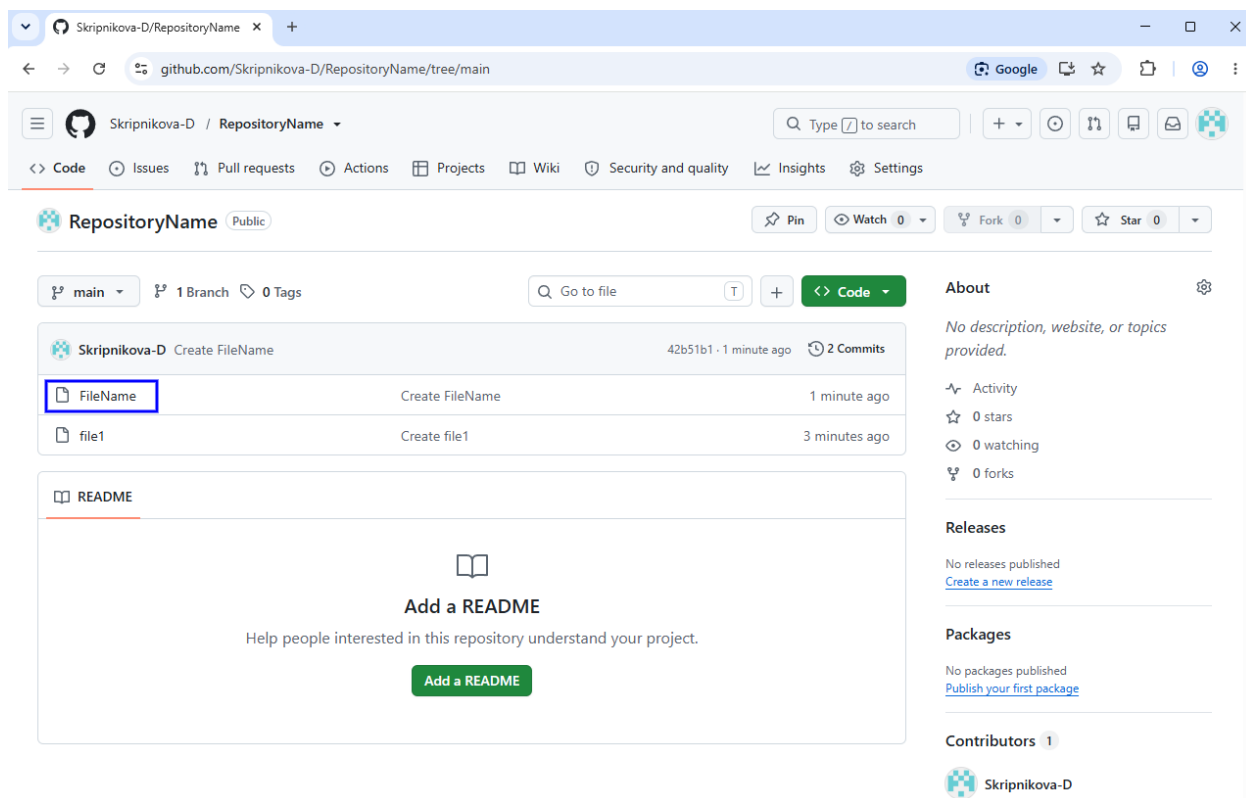


Рисунок 6 – Удаление файла. Часть 1

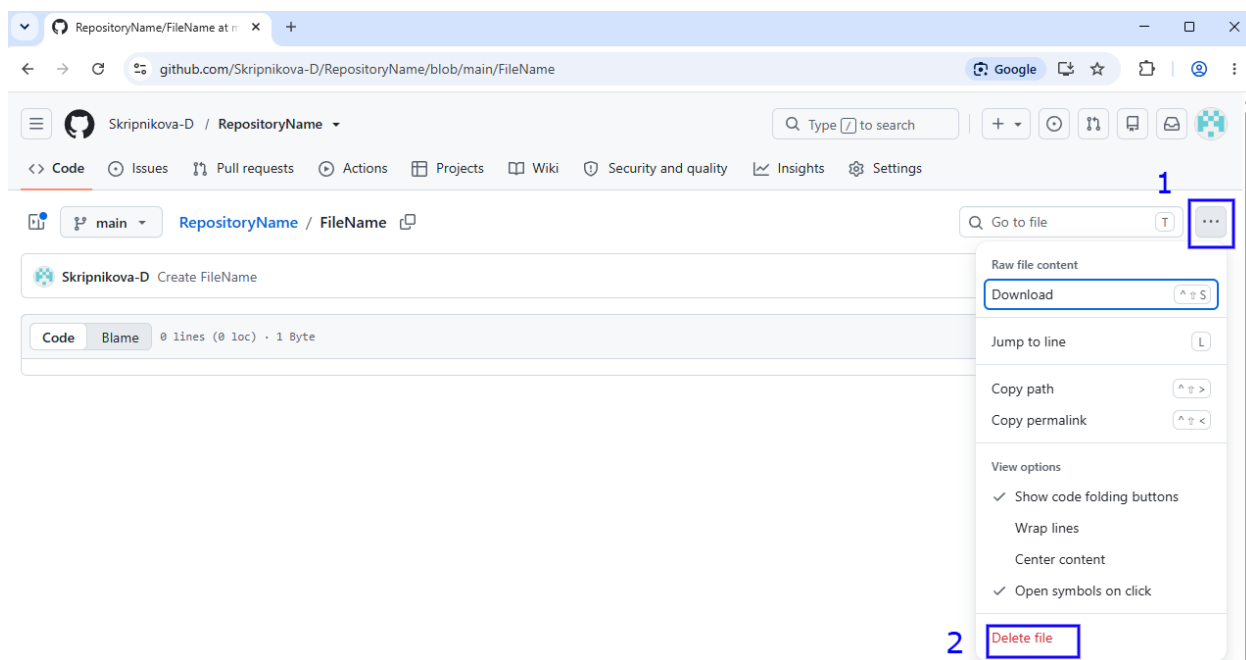


Рисунок 7 – Удаление файла. Часть 2

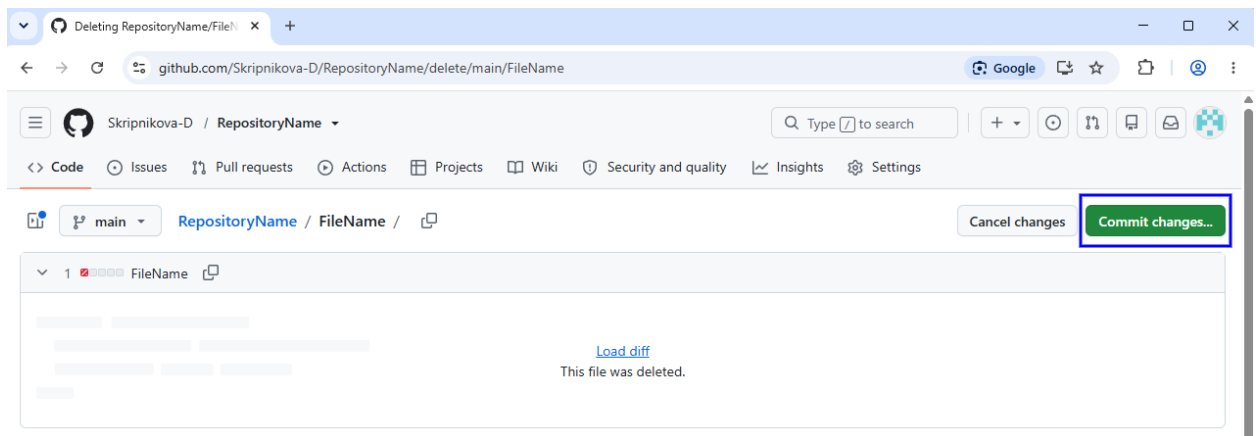


Рисунок 8 – Удаление файла. Часть 3

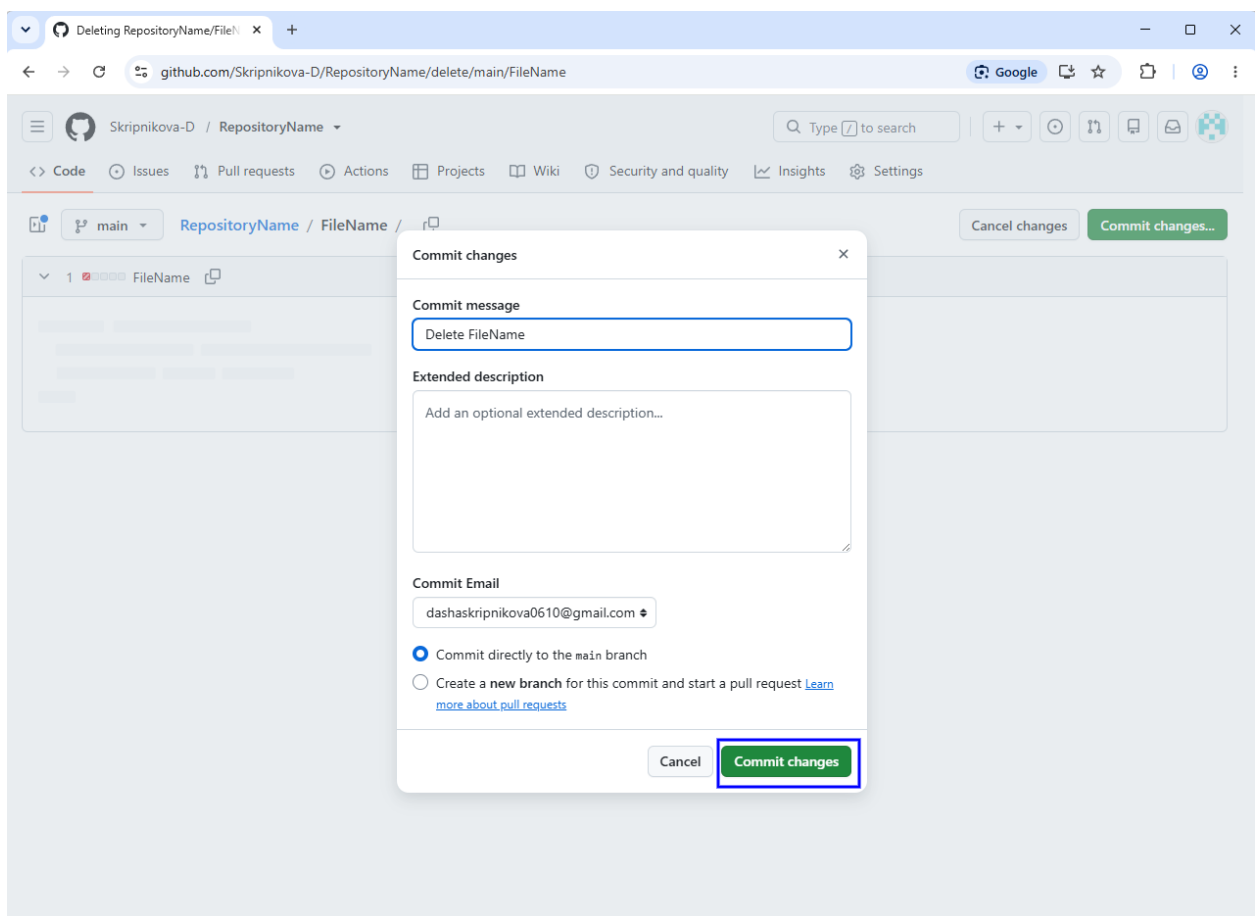


Рисунок 9 – Удаление файла. Часть 4

## 1.5 Создание ветки

Цель: проверить возможность создания новой ветки в репозитории.

Ожидаемый результат: новая ветка успешно создаётся и отображается в списке веток репозитория.

Шаги выполнения теста показаны на рисунках 10 - 12 (рис. 10, рис. 11, рис. 12)

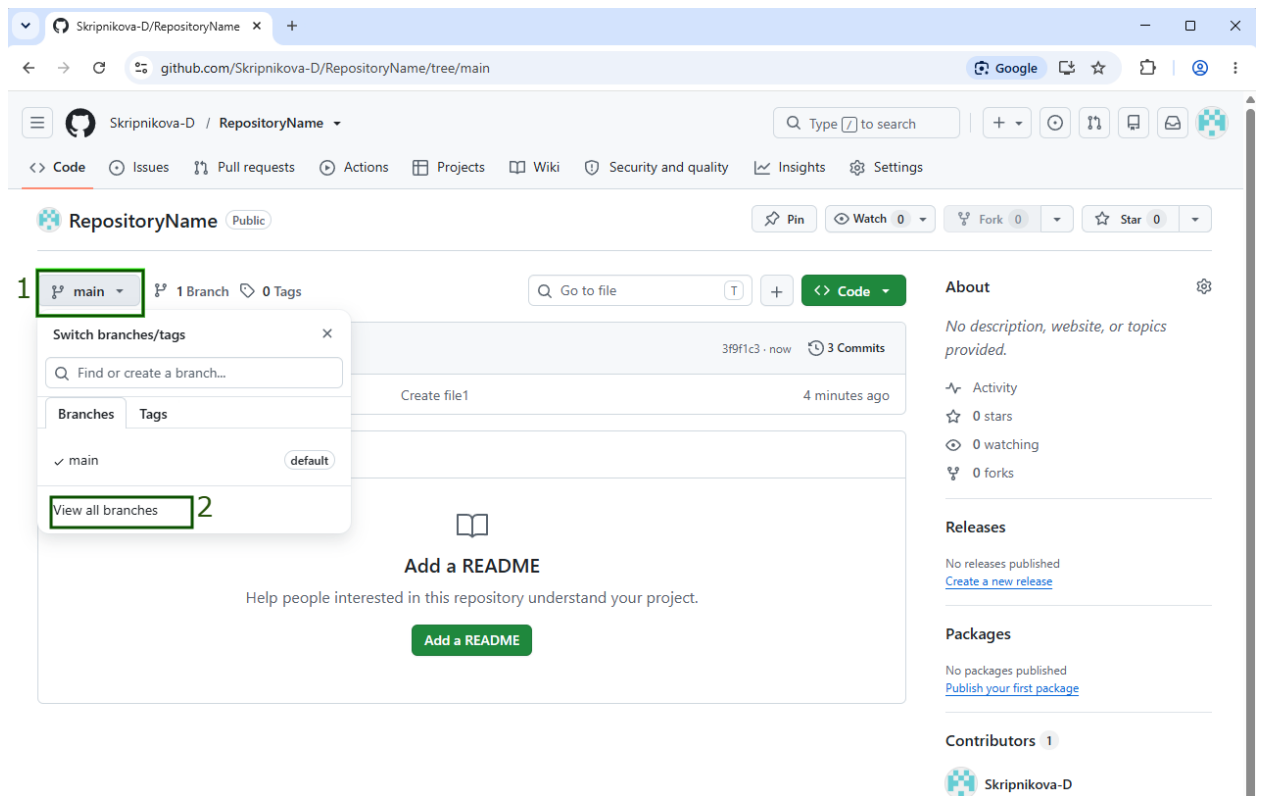


Рисунок 10 – Создание ветки. Часть 1

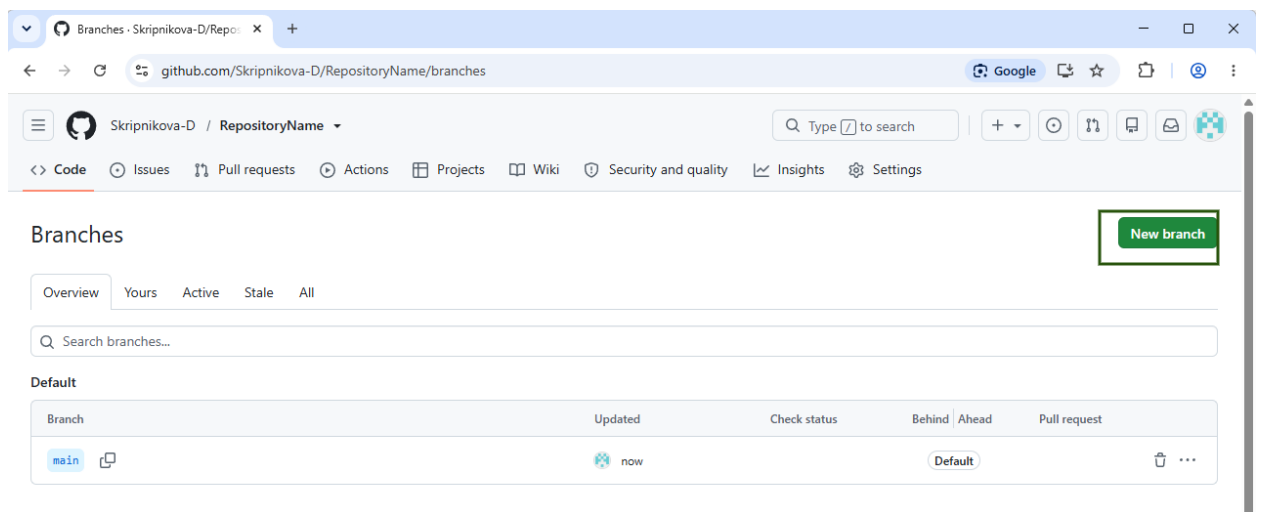


Рисунок 11 – Создание ветки. Часть 2

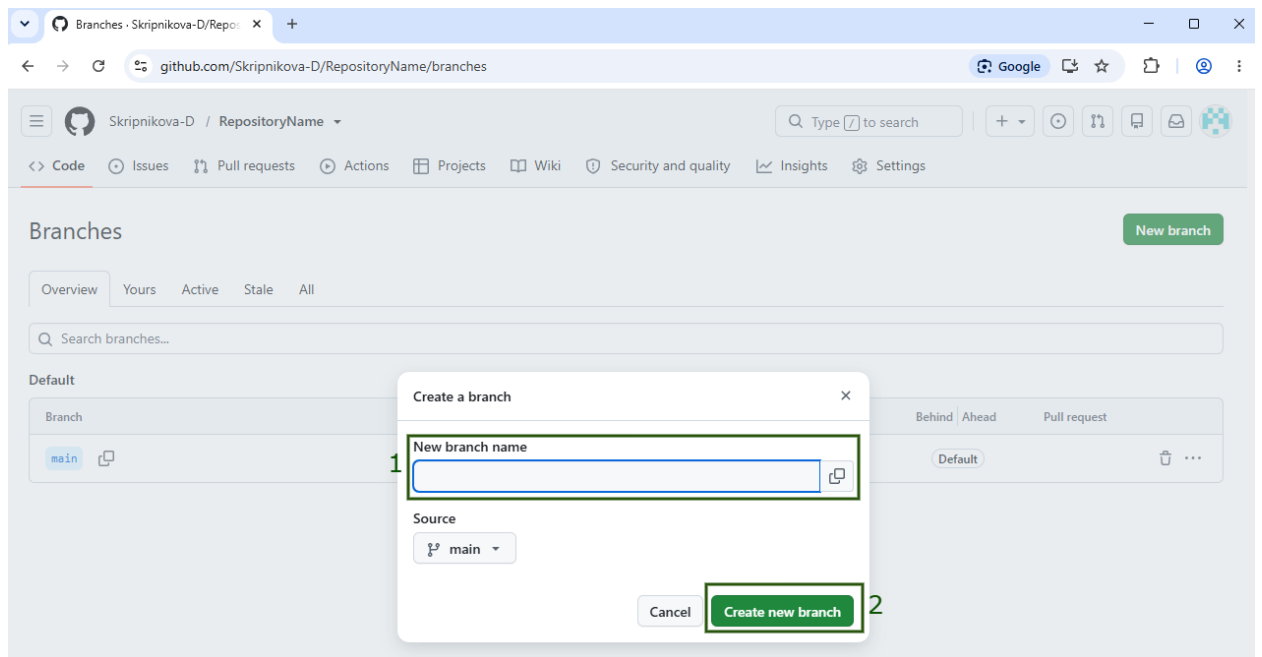


Рисунок 12 – Создание ветки. Часть 3

## 1.6 Удаление ветки

Цель: проверить возможность удаления ветки.

Ожидаемый результат: ветка удаляется и не отображается в списке веток.

Шаги выполнения теста показаны на рисунках 13 - 14 (рис. 13, рис. 14 )

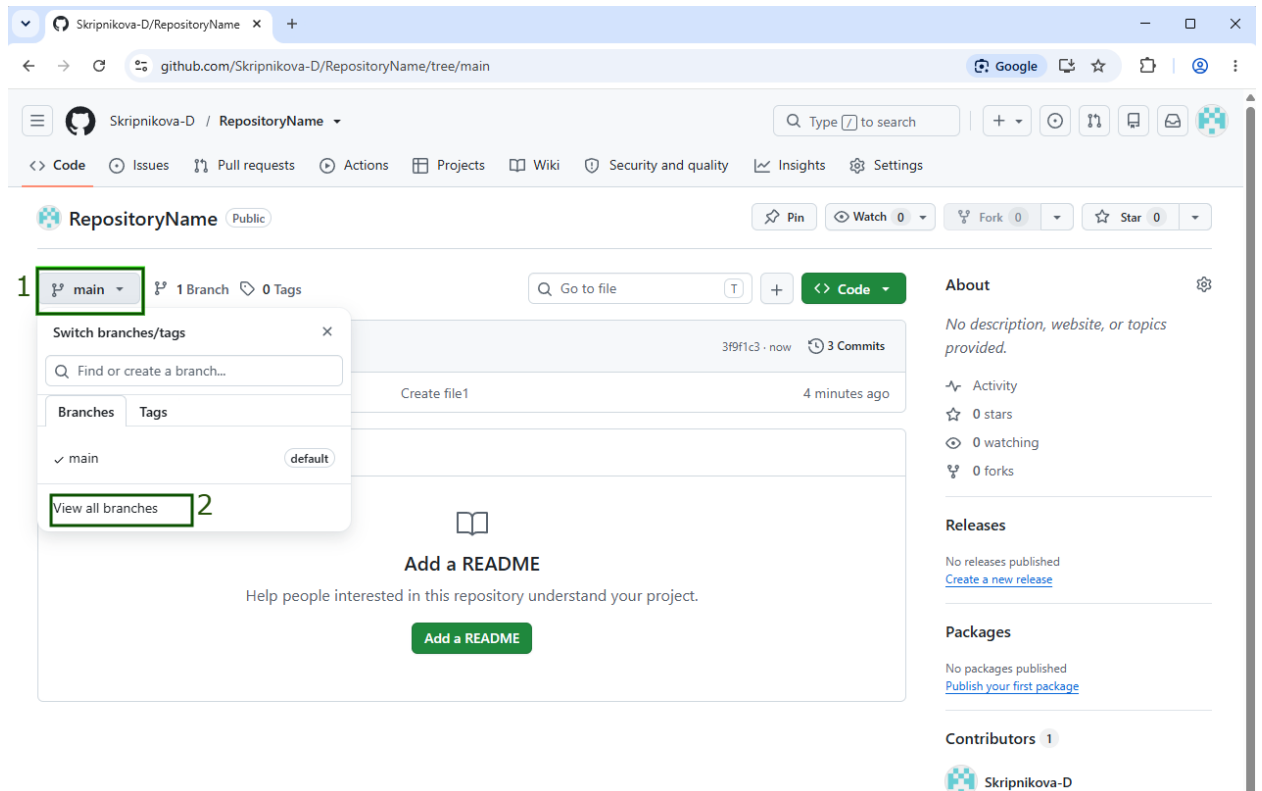


Рисунок 13 – Удаление ветки. Часть 1

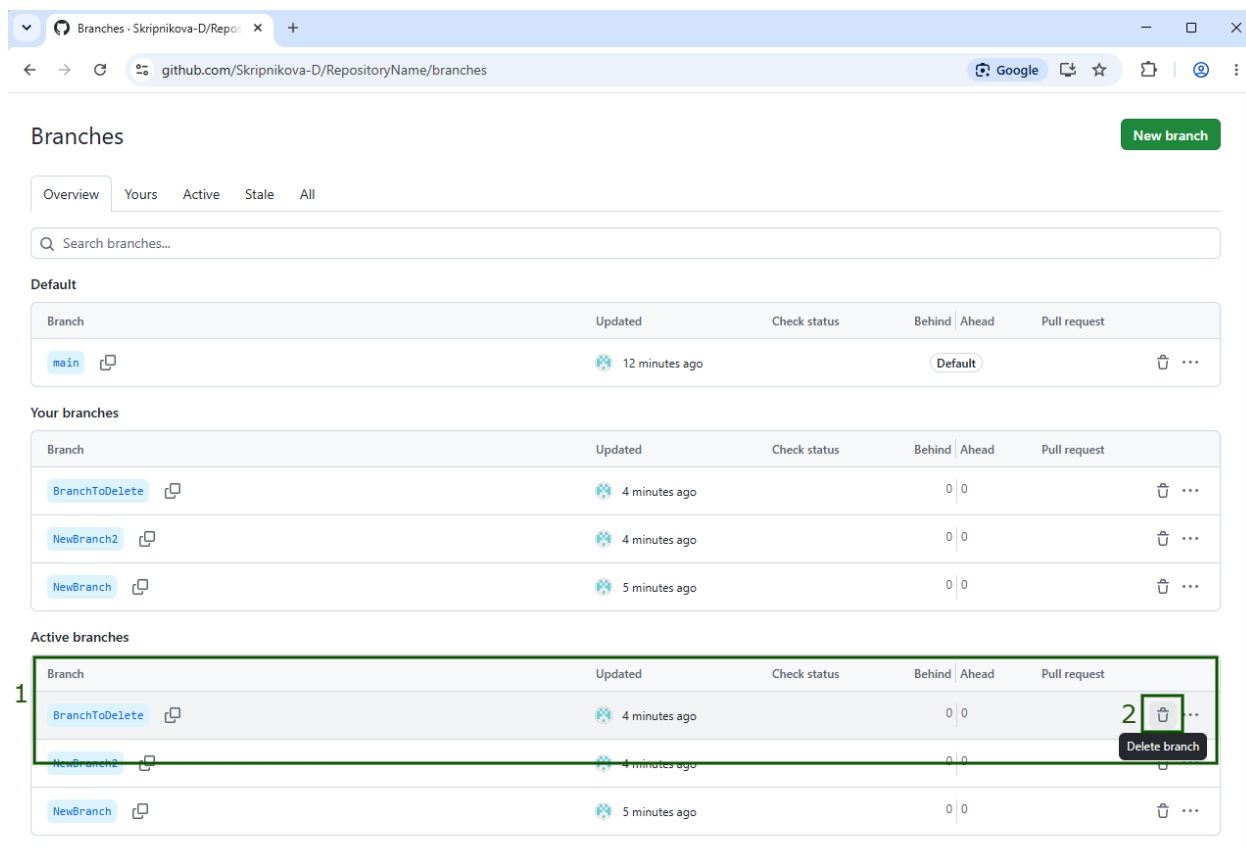


Рисунок 14 – Удаление ветки. Часть 2

## 1.7 Создание Pull Request

Цель: проверить процесс создания Pull Request для слияния изменений из одной ветки в другую.

Ожидаемый результат: Pull Request создаётся, открыт и отображается в списке Pull Request'ов.

Шаги выполнения теста показаны на рисунках 15 - 19 (рис. 15, рис. 16, рис. 17, рис. 18, рис. 19 )

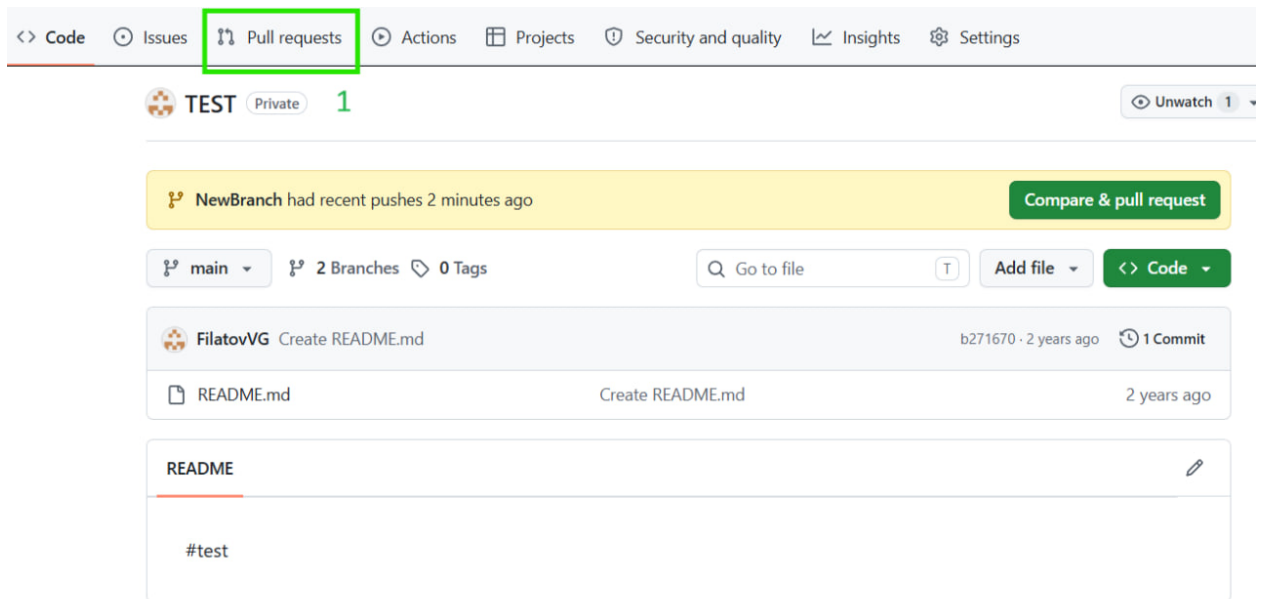


Рисунок 15 – Создание Pull Request. Часть 1

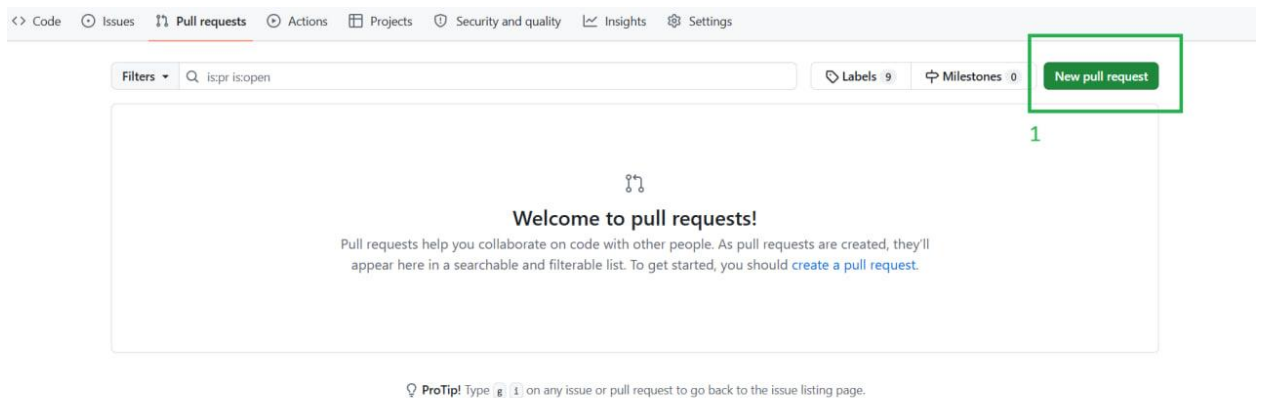


Рисунок 16 -- Создание Pull Request. Часть 2

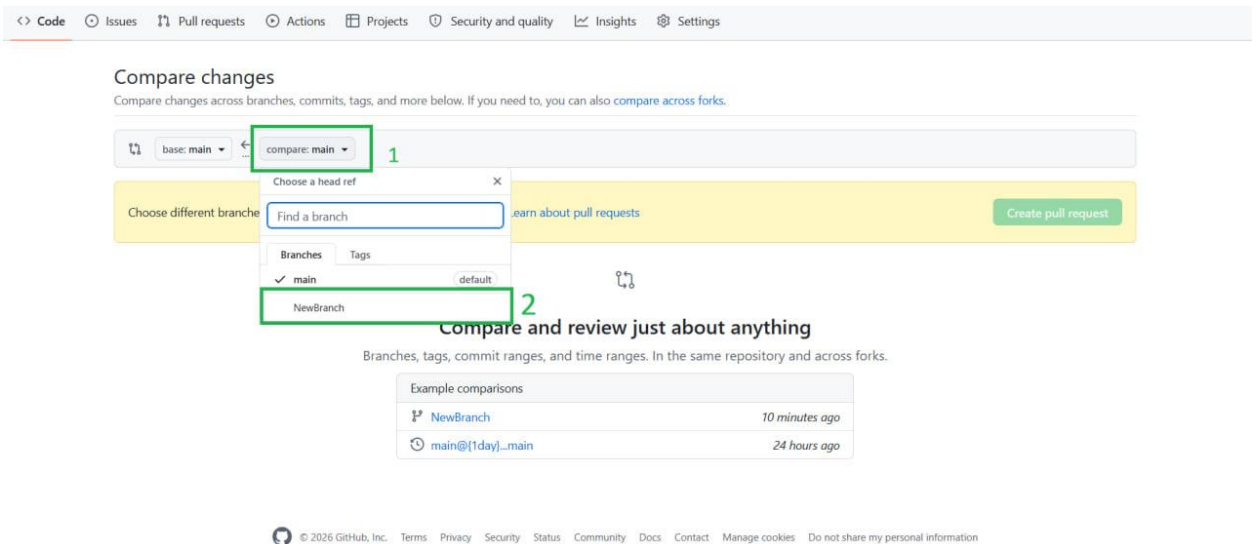
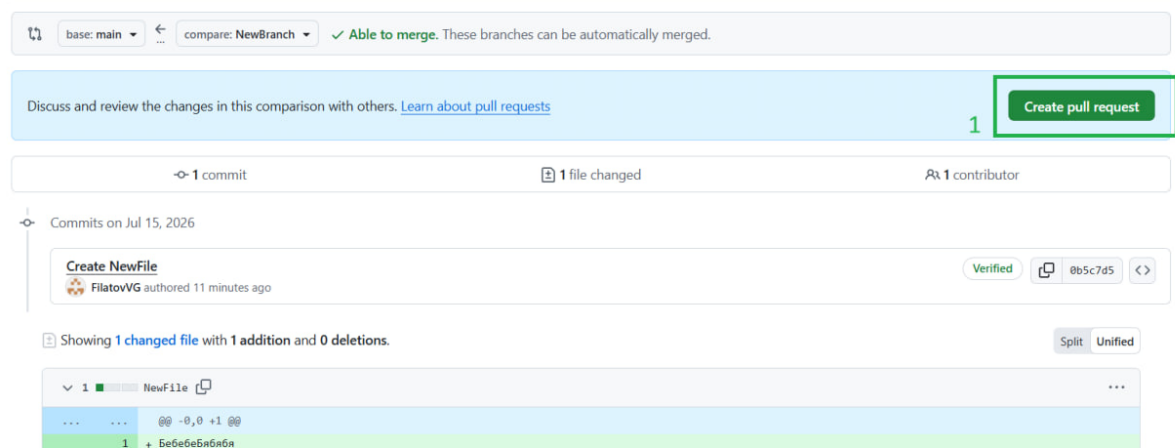


Рисунок 17 -- Создание Pull Request. Часть 3

### Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).



© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Рисунок 18 – Создание Pull Request. Часть 4

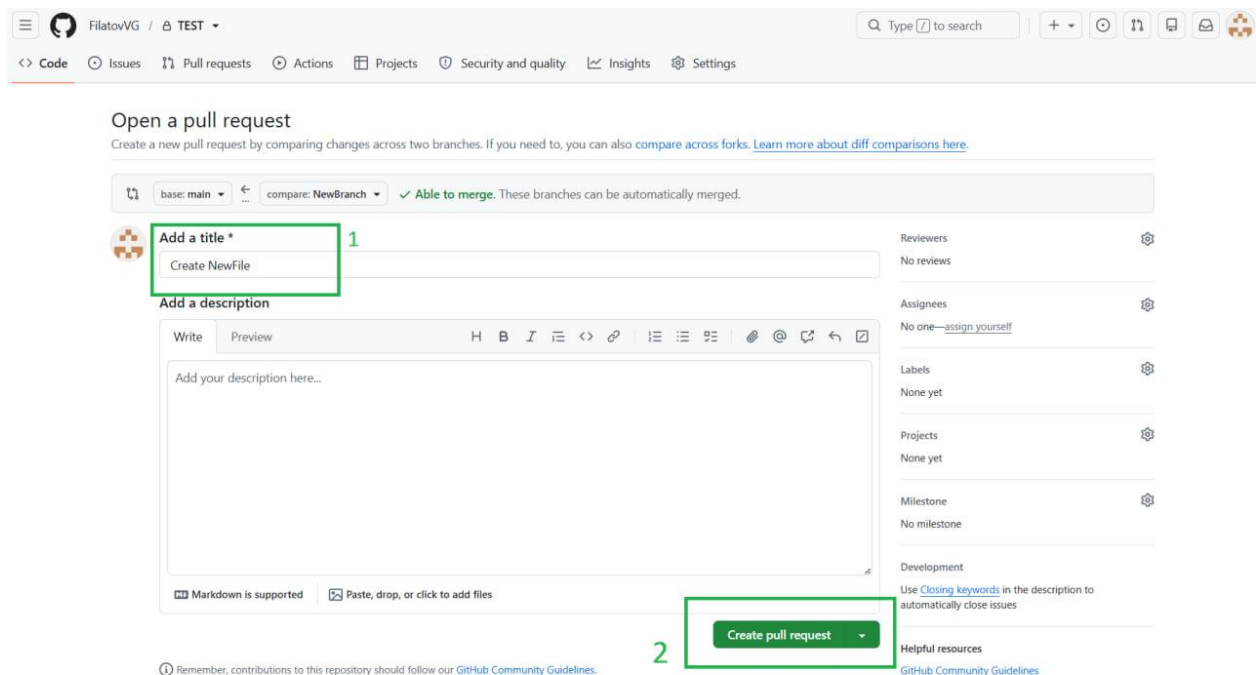


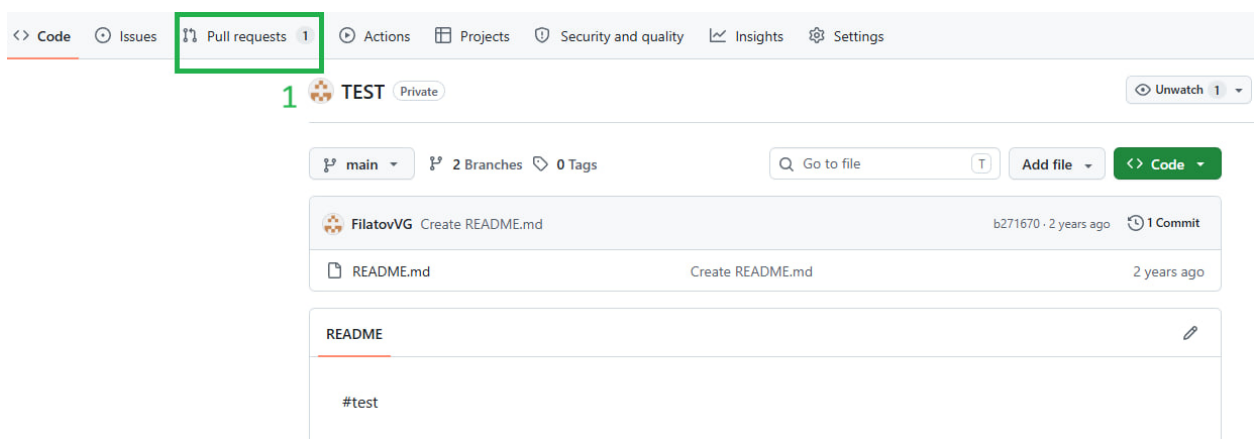
Рисунок 19 – Создание Pull Request. Часть 5

## 1.8 Заккрытие Pull Request

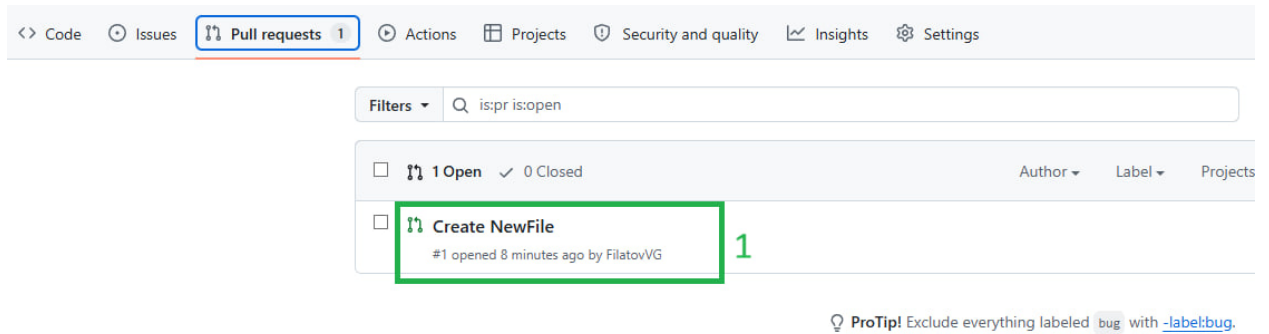
Цель: проверить возможность закрытия Pull Request без слияния.

Ожидаемый результат: Pull Request закрывается и приобретает статус «Closed».

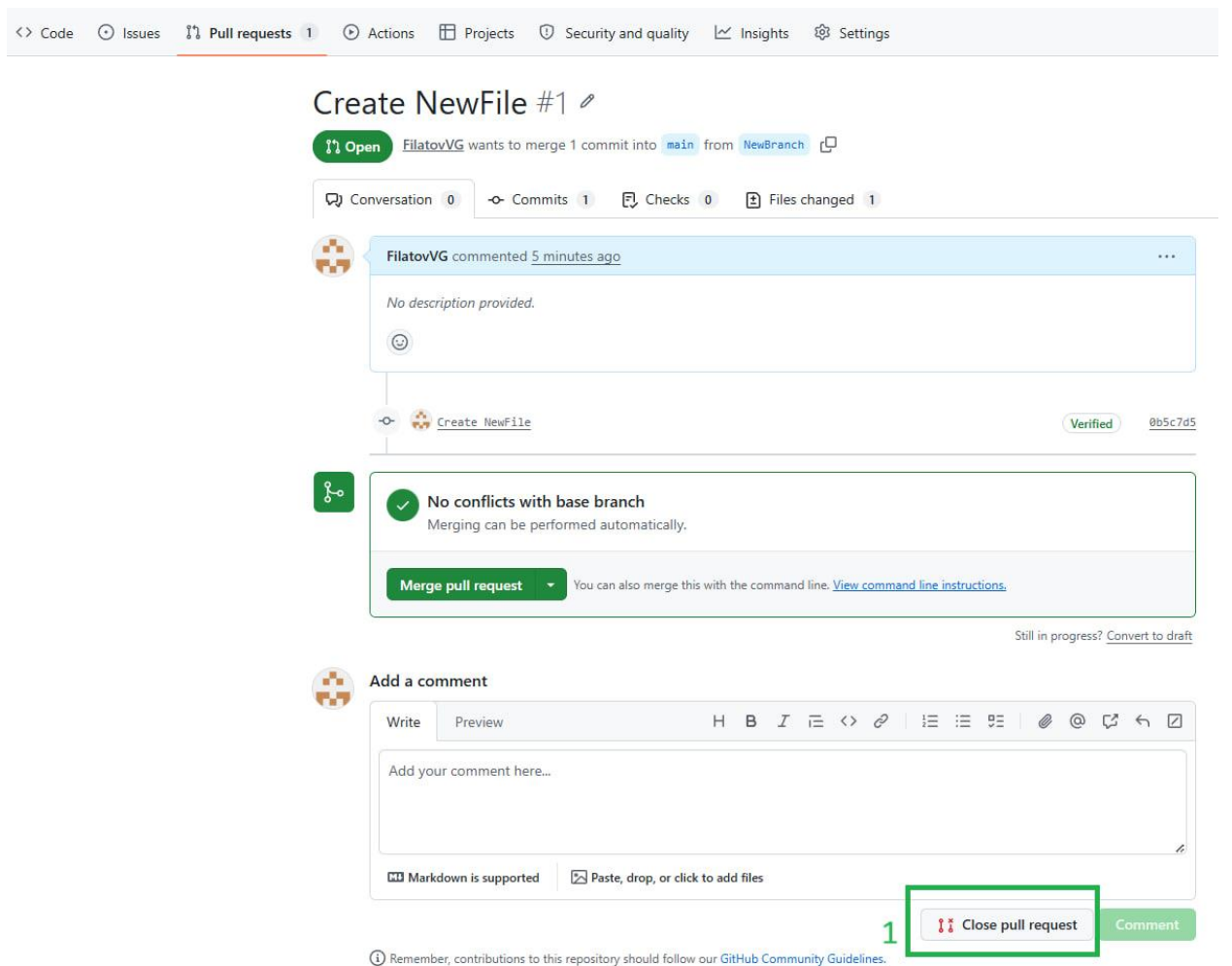
Шаги выполнения теста показаны на рисунках 20 - 22 (рис. 20, рис. 21 рис. 22)



## Рисунок 20 – Заккрытие Pull Request. Часть 1



## Рисунок 21 – Заккрытие Pull Request. Часть 2



## Рисунок 22 – Заккрытие Pull Request. Часть 3

### 1.9 Результат выполнения тестов

Результат выполнения тестов (рис. 23).

```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 04:49 min
[INFO] Finished at: 2026-07-16T11:37:15-04:00
[INFO] -----
```

Рисунок 23 - Подтверждение выполнения тестов

## 2 РЕЗУЛЬТАТЫ РАБОТЫ

### 2.1. Формулировка задачи 1 (Элементы)

В рамках задачи разработана иерархия классов для работы с веб-элементами на основе Selenide. Все классы наследуют базовую функциональность от `BaseElement`, которая обеспечивает поиск элементов по XPath и проверку их видимости.

#### `BaseElement`

Базовый абстрактный класс для всех элементов страницы.

Методы:

`isDisplayed()` - проверяет, отображается ли элемент на странице (с ожиданием до 5 секунд).

#### `Button`

Предназначен для работы с кнопками (`<button>`). Реализует интерфейс `Clickable`.

Методы:

- `click()` - клик по кнопке.
- `getText()` - получение текста кнопки.

Статические Методы поиска:

- `byId(String id)` - поиск по атрибуту `id`.
- `byContainsText(String text)` - поиск по части текста внутри кнопки.
- `byAriaLabel(String ariaLabel)` - поиск по `aria-label`.
- `byDataTestId(String testId)` - поиск по `data-testid`.

#### `Input`

Класс для работы с полями ввода (`<input>`).

Методы:

- `setValue(String text)` - ввод текста в поле.

Методы поиска:

- `byId(String id)` - по `id`.
- `byAriaLabel(String ariaLabel)` - по `aria-label`.
- `byLabel(String labelText)` - по тексту связанного `label`.

## Link

Класс для работы со ссылками (<a>). Реализует Clickable.

Методы:

- click() - клик по ссылке.

Методы поиска:

- byHref(String href) - по полному href.
- byContainsText(String text) - по части текста.
- byAriaLabel(String ariaLabel) - по aria-label.
- byHrefWithLocation(String href, String location) - по href с уточнением по data-hydro-click.
- byDataTabItem(String tabItem) - по data-tab-item.

## CodeMirrorInput

Класс для работы с редактором кода CodeMirror (поле содержимого файла).

Методы:

- setValue(String text) - ввод текста в редактор (поддерживает разные варианты DOM-структуры).

Методы поиска:

- byClass(String className) - поиск по CSS-классу.

## DropDownItem

Класс для пунктов выпадающего списка (<li> внутри <ul role="menu">).

Реализует Clickable.

Методы:

- click() - выбор пункта.

Методы поиска:

- byMenuAndText(String menuRole, String text) - поиск по роли меню и тексту пункта.

## SelectMenu

Компонент для выбора веток в интерфейсе Pull Request.

Методы:

- `select(String branchName)` - выбор ветки по названию

#### Summary

Класс для работы с элементами `<summary>` (раскрывающиеся списки).

Методы:

- `click()` - раскрытие/закрытие списка.
- `getText()` - получение текста.

Методы поиска:

- `byContainsText(String text)` - поиск по части текста.

#### BranchTable

Составной элемент для работы с таблицами (например, список веток или файлов).

Статические методы:

- `deleteBranch(String tableName, String branchName)` - удаление ветки по названию таблицы и имени ветки.
- `clickFileLink(String tdClass, String fileName)` - клик по ссылке на файл по классу ячейки и title.

#### Clickable

Интерфейс, объединяющий все кликабельные элементы.

Метод:

- `click()` - выполнение клика.

## 2.2. Формулировка задачи 2 (Страницы)

#### BasePage.java

Базовый класс для всех страниц.

Методы:

`isLoading()` — проверяет, загружена ли страница (проверка видимости body)

#### BaseModal.java

Базовый класс для всех модальных окон.

Методы:

`waitForOpen()` — ожидает открытия модального окна

`waitForClose()` — ожидает закрытия модального окна

`BranchesPage.java`

Страница со списком веток репозитория.

Методы:

`clickNewBranchButton()` — нажимает кнопку "New branch"

`setBranchName(String branchName)` — устанавливает имя новой ветки

`clickCreateNewBranch()` — нажимает кнопку "Create new branch"

`deleteBranch(String branchName)` — удаляет ветку по имени

`clickBranch(String branchName)` — кликает на ветку,

возвращает `RepositoryPage`

`isBranchExists(String branchName)` — проверяет существование ветки

`CommitModal.java`

Модальное окно коммита изменений.

Методы:

`confirm()` — подтверждает коммит, возвращает `RepositoryPage`

`CommitPage.java`

Страница коммита.

Методы:

`confirm()` — подтверждает коммит, возвращает `RepositoryPage`

`CreateFilePage.java`

Страница создания нового файла.

Методы

`setFileName(String fileName)` — устанавливает имя файла

`setFileContent(String content)` — устанавливает содержимое файла

`clickCommitChangesButton()` — нажимает кнопку "Commit changes...", открывает `CommitModal`

`FilePage.java`

Страница просмотра файла.

Методы:

`clickMoreOptions()` — открывает меню трех точек

`selectDeleteFile()` — выбирает опцию "Delete file", возвращает `CommitPage`

`MainPage.java`

Главная страница GitHub после авторизации.

Методы:

`clickNewButton()` — нажимает кнопку "New", возвращает `NewRepositoryPage`

`openRepository(String repoName)` — открывает репозиторий по имени

`NewPullRequestPage.java`

Страница создания нового Pull Request.

Методы:

`selectBaseBranch(String branchName)` — выбирает base ветку для Pull Request

`selectCompareBranch(String branchName)` — выбирает compare ветку для Pull Request

`clickCreatePullRequestButton()` — нажимает кнопку Create pull request (первый этап создания PR)

`setPullRequestTitle(String title)` — устанавливает заголовок Pull Request

`confirmCreatePullRequest()` — подтверждает создание Pull Request, возвращает `PullRequestsPage`

NewRepositoryPage.java

Страница создания нового репозитория.

Методы:

setRepositoryName(String name) — устанавливает имя репозитория

selectVisibility() — открывает меню выбора видимости

selectPublicVisibility() — выбирает публичную видимость

selectPrivateVisibility() — выбирает приватную видимость

clickCreateRepositoryButton() — нажимает кнопку создания репозитория,  
возвращает RepositoryPage

PullRequestPage.java

Страница конкретного Pull Request.

Методы:

clickClosePullRequest() — нажимает кнопку Close pull request

isPullRequestClosed(String prName) — проверяет, закрыт ли Pull Request

PullRequestsPage.java

Страница со списком Pull Requests.

Методы:

clickNewPullRequestButton() — нажимает кнопку "New pull request",  
возвращает NewPullRequestPage

clickPullRequest(String prName) — кликает на Pull Request по имени,  
возвращает PullRequestPage

isPullRequestInList(String title) — проверяет, отображается ли Pull  
Request в списке

RepositoryPage.java

Страница репозитория.

Методы:

goToCodeTab() — переходит на вкладку Code

`goToPullRequestsTab()` — переходит на вкладку Pull requests, возвращает `PullRequestsPage`

`clickAddFileButton()` — нажимает кнопку "Add file"

`selectCreateNewFileOption()` — выбирает опцию "Create new file", возвращает `CreateFilePage`

`clickViewAllBranches()` — нажимает "View all branches", возвращает `BranchesPage`

`clickOnFile(String fileName)` — кликает на файл по имени, возвращает `FilePage`

`isFileExists(String fileName)` — проверяет, существует ли файл в репозитории

`getRepositoryName()` — получает имя репозитория

`isPrivate()` — проверяет, является ли репозиторий приватным

`isRepositoryInList(String repoName)` — проверяет, отображается ли репозиторий в списке

`openRepository(String repoName)` — статический метод, открывает репозиторий по имени

### 2.3. Формулировка задачи 3 (Тесты)

`RepoTest` (тестирование управления репозиториями)

Класс содержит тесты для проверки создания репозитория с различными настройками видимости.

- `createPublicRepositoryTest()` – тест создания публичного репозитория. Выполняет последовательность действий: переход к форме создания, ввод уникального имени, выбор публичного типа видимости, подтверждение создания и проверка загрузки страницы репозитория с корректным именем.

- `createPrivateRepositoryTest()` – тест создания приватного репозитория. Аналогичен предыдущему, но выбирает приватный тип видимости. Дополнительно проверяет корректное отображение статуса приватности и наличие созданного репозитория в общем списке.

Для обоих тестов используется вспомогательный метод `createRepository(String repoName, boolean isPrivate)`, инкапсулирующий общую логику создания репозитория.

#### BranchTests (тестирование управления ветками)

Класс содержит тесты для проверки полного жизненного цикла веток в репозитории.

- `createNewBranchTest()` – тест создания новой ветки. Открывает целевой репозиторий, переходит на страницу управления ветками, иницирует создание новой ветки с уникальным именем, подтверждает операцию и проверяет появление ветки в общем списке.

- `deleteBranchTest()` – тест удаления ветки. Переходит на страницу управления ветками, выполняет удаление ранее созданной ветки, обновляет страницу и проверяет отсутствие ветки в списке.

#### FileTest (тестирование управления файлами)

Класс содержит тесты для проверки создания и удаления файлов в репозитории. Тесты выполняются последовательно с использованием аннотации `@Order`.

- `createFileTest()` – тест создания файла. Открывает целевой репозиторий, через меню "Add file" выбирает опцию создания нового файла, вводит уникальное имя и содержимое, подтверждает коммит через модальное окно и проверяет появление файла в списке репозитория.

- `deleteFileTest()` – тест удаления файла. Открывает репозиторий, проверяет существование файла, переходит на его страницу, через меню "More options" выбирает опцию удаления, подтверждает операцию и проверяет отсутствие файла в списке.

#### PullRequestTests (тестирование управления Pull Request'ами)

Класс содержит тесты для проверки создания и закрытия запросов на слияние. Тесты выполняются последовательно.

- `createPullRequestTest()` – тест создания Pull Request'a. Открывает репозиторий, переходит к существующей ветке, создает новый файл в этой ветке, переходит в раздел управления Pull Request'ами, инициирует создание нового запроса с выбором базовой ветки и ветки для сравнения, подтверждает создание и проверяет наличие созданного запроса в общем списке.

- `closePullRequestTest()` – тест закрытия Pull Request'a. Переходит на страницу созданного запроса, выбирает операцию закрытия и проверяет корректное отображение статуса "Closed" на странице запроса.

#### Общее описание архитектуры

Все тесты наследуются от базового класса `'BaseTest'`, который предоставляет:

- `loginViaCookies()` – метод для авторизации через cookies, ускоряющий выполнение тестов;

- поле `RUN_ID` – генератор уникальных идентификаторов для имен создаваемых объектов, предотвращающий конфликты при параллельном запуске.

Тесты используют страницы, реализованные по паттерну Page Object Model, что обеспечивает разделение логики тестов и деталей реализации элементов страниц. Каждый тест включает шаги по навигации, выполнению действий и проверке ожидаемых результатов с использованием Assertions.

UML диаграмму реализованных классов можно увидеть на рис. 24

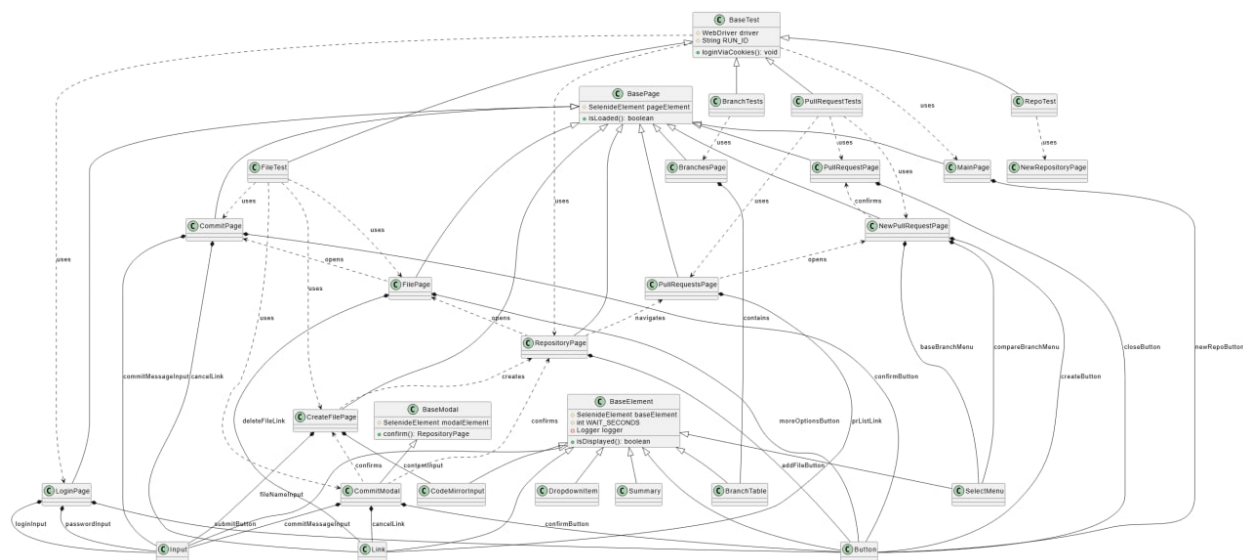


Рисунок 24 - UML диаграмма классов

### 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В рамках проекта моей задачей была разработка Page Object классов для автоматизации тестирования GitHub. Page Object - это классы, которые представляют отдельные страницы или части интерфейса. Каждый такой класс содержит все элементы конкретной страницы и методы для взаимодействия с ними. Это позволяет отделить логику тестов от деталей реализации и упростить поддержку автотестов.

#### 3.1 Базовый класс BasePage

Был создан базовый класс BasePage, от которого наследуются все страницы. Он содержит общие методы:

isLoading() — проверяет, загружена ли страница (проверка видимости body)

Этот класс можно расширять, добавляя новые общие методы для всех страниц.

#### 3.2 Какие страницы были реализованы

MainPage.java

Главная страница GitHub после авторизации.

Методы:

clickNewButton() — нажимает кнопку "New" для создания нового репозитория, возвращает страницу NewRepositoryPage

openRepository(String repoName) — открывает репозиторий по имени, возвращает страницу RepositoryPage

NewRepositoryPage.java

Страница создания нового репозитория.

Методы:

setRepositoryName(String name) — устанавливает имя репозитория

selectVisibility() — открывает меню выбора видимости

selectPublicVisibility() — выбирает публичную видимость

selectPrivateVisibility() — выбирает приватную видимость

`clickCreateRepositoryButton()` — нажимает кнопку создания репозитория, возвращает страницу `RepositoryPage`

`RepositoryPage.java`

Страница репозитория.

Методы:

`goToCodeTab()` — переходит на вкладку `Code`

`goToPullRequestsTab()` — переходит на вкладку `Pull requests`, возвращает `PullRequestsPage`

`clickAddFileButton()` — нажимает кнопку "Add file"

`selectCreateNewFileOption()` — выбирает опцию "Create new file", возвращает `CreateFilePage`

`clickViewAllBranches()` — нажимает "View all branches", возвращает `BranchesPage`

`clickOnFile(String fileName)` — кликает на файл по имени, возвращает `FilePage`

`isFileExists(String fileName)` — проверяет, существует ли файл в репозитории

`getRepositoryName()` — получает имя репозитория

`isPrivate()` — проверяет, является ли репозиторий приватным

`isRepositoryInList(String repoName)` — проверяет, отображается ли репозиторий в списке

`openRepository(String repoName)` — статический метод для открытия репозитория по имени

`CreateFilePage.java`

Страница создания нового файла.

Методы:

`setFileName(String fileName)` — устанавливает имя файла

`setContent(String content)` — устанавливает содержимое файла

clickCommitChangesButton() — нажимает кнопку "Commit changes...", открывает CommitModal

FilePage.java

Страница просмотра файла.

Методы:

clickMoreOptions() — открывает меню трех точек

selectDeleteFile() — выбирает опцию "Delete file", возвращает CommitPage

CommitPage.java

Страница коммита (подтверждения изменений).

Методы:

confirm() — подтверждает коммит, возвращает страницу RepositoryPage  
CommitModal.java

Модальное окно коммита изменений.

Методы:

confirm() — подтверждает коммит, возвращает страницу RepositoryPage

BranchesPage.java

Страница со списком веток репозитория.

Методы:

clickNewBranchButton() — нажимает кнопку "New branch"

setBranchName(String branchName) — устанавливает имя новой ветки

clickCreateNewBranch() — нажимает кнопку "Create new branch"

deleteBranch(String branchName) — удаляет ветку по имени

clickBranch(String branchName) — кликает на ветку, возвращает RepositoryPage

isBranchExists(String branchName) — проверяет существование ветки

NewPullRequestPage.java

Страница создания нового Pull Request.

Методы:

selectBaseBranch(String branchName) — выбирает base ветку для Pull Request

selectCompareBranch(String branchName) — выбирает compare ветку для Pull Request

clickCreatePullRequestButton() — нажимает кнопку Create pull request (первый этап создания PR)

setPullRequestTitle(String title) — устанавливает заголовок Pull Request

confirmCreatePullRequest() — подтверждает создание Pull Request, возвращает PullRequestsPage

PullRequestsPage.java

Страница со списком Pull Requests.

Методы:

clickNewPullRequestButton() — нажимает кнопку "New pull request", возвращает NewPullRequestPage

clickPullRequest(String prName) — кликает на Pull Request по имени, возвращает PullRequestPage

isPullRequestInList(String title) — проверяет, отображается ли Pull Request в списке

PullRequestPage.java

Страница конкретного Pull Request.

Методы:

clickClosePullRequest() — нажимает кнопку Close pull request

isPullRequestClosed(String prName) — проверяет, закрыт ли Pull Request

### 3.3 Использование элементов

При создании страниц использовались готовые элементы (Button, Input, Link, DropdownItem, CodeMirrorInput и другие). Это позволило быстро и единообразно описывать элементы интерфейса без дублирования кода поиска.

Пример:

```
private final Button newBranchButton = Button.byContainsText("New  
branch");  
  
private final Input branchNameInput = Input.byLabel("New branch name");  
private final Link pullRequestsTab = Link.byHref("/pulls");
```

### **3.4 Использование элементов**

Методы, которые выполняют действия и переходят на другую страницу, возвращают объект соответствующей страницы. Это обеспечивает плавную цепочку действий в тестах.

#### 4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на отлично.  
Отзыв руководителя с оценкой (см. рис. 25).

##### Отзыв

##### о прохождении учебной практики

Дунченко Никита Сергеевич, студент группы 4345 второго курса бакалавриата Санкт-Петербургского электротехнического университета им В.И. Ульянова (Ленина) “ЛЭТИ”, проходил учебную практику по теме “UI-тестирование” с 25.06.2025 по 08.07.2025 на кафедре МО ЭВМ.

Во время практики Дунченко Никита Сергеевич выполнил все поставленные задачи, продемонстрировал владение языком программирования JAVA, умение планировать разработку и работать в команде.

По результатам учебной практики Дунченко Никита Сергеевич заслуживает оценки **отлично**.

Подпись руководителя практики:

|                            |                                                                                     |               |
|----------------------------|-------------------------------------------------------------------------------------|---------------|
| Ассистент кафедры<br>МОЭВМ |  | Шевелева А.М. |
|                            |                                                                                     | 17.07.2026    |

Рисунок 25 - Отзыв руководителя

## ЗАКЛЮЧЕНИЕ

В ходе выполнения летней учебной практики по направлению «UI-тестирование» была достигнута поставленная цель - разработан набор автоматизированных UI-тестов для веб-приложения (GitHub) на языке Java. В рамках работы был реализован полноценный тестовый проект, построенный на паттерне Page Object Model, что обеспечило его удобство для командной разработки.

Поставленные задачи были решены в полном объёме. Разработано 8 автоматизированных тестов, покрывающих ключевые функции выбранного блока тестирования: создание репозитория с различными настройками видимости, работу с файловой структурой (создание и удаление файлов), управление ветками (создание и удаление), а также создание и закрытие Pull Request'ов.

Значительное внимание было уделено созданию устойчивых XPath-выражений, позволяющих надёжно идентифицировать элементы на странице.

Настроенное логирование через Log4j позволяет автоматически фиксировать все действия Selenide, что упрощает отладку и анализ результатов выполнения тестов.

В ходе работы были освоены навыки командной разработки, работы с системой контроля версий Git, написания автотестов на Java. Получен практический опыт применения паттерна Page Object Model, работы с XPath и настройки логирования. Результаты работы подтверждены успешным прохождением всех тестов, что свидетельствует о корректности реализованных проверок.

## **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Java. Базовый курс [Электронный ресурс]. URL: <https://stepik.org/course/187/info> (дата обращения: 08.07.2026).
2. Apache Maven Project [Электронный ресурс]. URL: <https://maven.apache.org> (дата обращения: 09.07.2026).
3. JUnit [Электронный ресурс]. URL: <https://junit.org> (дата обращения: 11.07.2026).
4. Selenide [Электронный ресурс]. URL: <https://selenide.org> (дата обращения: 09.07.2026).
5. Github-Testing-Project [Электронный ресурс] : летняя учебная практика 2026, UI-Тестирование GitHub. URL: <https://github.com/Skripnikova-D/Github-Testing-Project> (дата обращения: 12.07.2026).